

AMR, Life Expectancy, and Intervention Impact

A complete history of the project: pipeline, integrity system, and dashboard

Repository: <https://github.com/enchill-svg/amr-vivli-2026> — public, MIT-licensed **As of:** 2026-07-22. Local `main` and `origin/main` are identical (0 ahead / 0 behind); working tree clean. 46 commits, `4227ed6` (2026-07-06) through `b0ef342` (2026-07-21), all pushed. **Purpose of this document:** a ground-up, plain-language account of everything this project currently does — the 30-stage analysis pipeline, the evidence-gate integrity system that keeps it honest, every one of the 42 dashboard route files, and how they fit together — with the reasoning behind the major decisions and the bugs that were found and fixed along the way. Every number below is read directly from a file that exists in the repository right now; every code claim cites the file it comes from.

1. What this project is, in one paragraph

This is a submission to the **Vivli AMR Surveillance Data Challenge 2026** (AMR Register Data Request ID **00013346**). It has two halves that have to agree with each other: a **30-stage Python analysis pipeline** (`analysis/`) that turns raw antimicrobial-resistance surveillance data into gated, honesty-checked statistical outputs, and a **React/TypeScript web dashboard** (`dashboard/`) that presents those outputs to a judge or visitor. The organizing idea, stated in the project's own README, is: "*We would rather show an empty ranked intervention list than rank a confounded estimate.*" Concretely, that means every number the dashboard shows the public carries a `quality_gate` label — `pass`, `bounds_only`, or `withhold` — and the dashboard is wired to respect that label rather than just displaying whatever a column contains.

2. Where it started: two separate projects that became one

The repository's first commit (`4227ed6`, 2026-07-06) is a bare pipeline skeleton — no dashboard at all. Over the next four days the team built out AMR preprocessing (`c5ce6fe`), EUCAST breakpoint / MIC parsing (`14d9d17`), and a rewritten pipeline README (`9896f7c`, `72ae251`). This was a pure data-science effort: no UI existed yet.

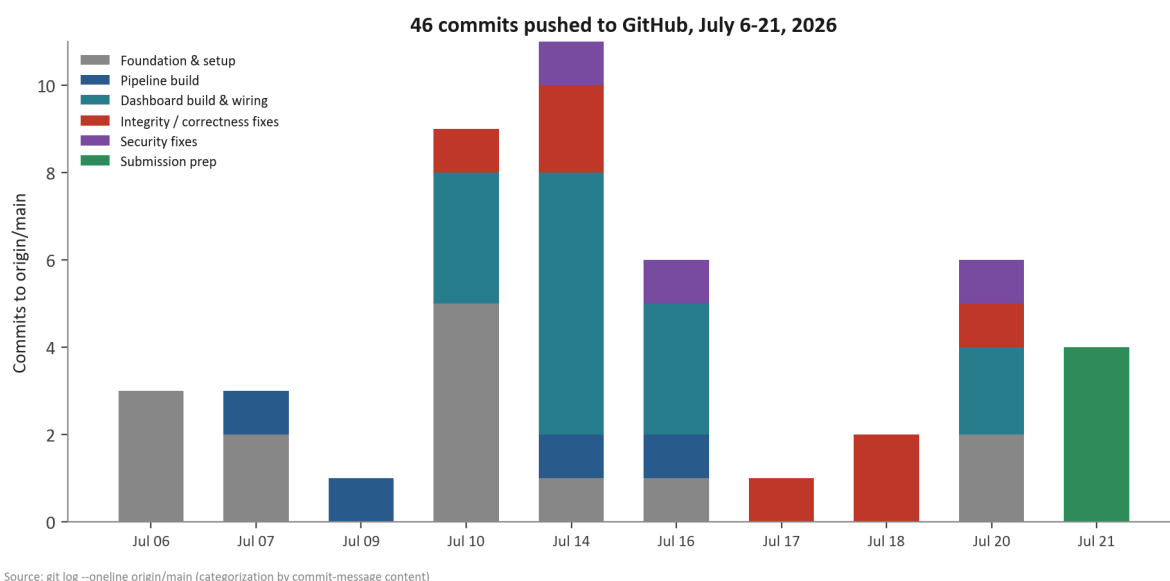
The dashboard came from somewhere else. `dashboard/.lovable/project.json` and `dashboard/.lovable/plan.md` (read directly) show the frontend originated as a **Lovable.dev-generated product called "ViralTrack-Afrika"** — a viral outbreak / wastewater-genomic-surveillance monitoring template, complete with pages for genomics, biosecurity, variant tracking, environmental monitoring, and a training academy. None of that subject matter is AMR. It was a template repurposed, not a dashboard purpose-built for this data.

The merge happened on **2026-07-10**, in a single dense day of commits: `d4aae99` ("Integrate one-pipeline analysis, gated publish bundle, and dashboard wiring") wired the two halves together for the first time, followed immediately by `23dc882` (removing ungated ranking tables that had leaked into the public folder), `48d92fe` / `611b44f` / `c521135` (pulling internal planning docs out of the public repo), `ab80dfb` (wiring dashboard routes to the newly published bundle), `21adf46` (fixing a JSON serialization bug where `NaN` values broke the bundle), and `b8f784b` (rewriting the top-level README to describe the merged product honestly). That's seven commits in one day — the moment the project stopped being "a pipeline" and "a demo dashboard" and became one product.

The rebrand from ViralTrack language to AMR language wasn't instantaneous, though. It took until **2026-07-14** (10afdd7, "Replace ViralTrack branding with AMR Life Expectancy Intelligence") for the product's own name to catch up with what it had become — four days during which the merged app still visually announced itself as a viral-surveillance tool while actually running AMR statistics underneath.

This history matters for one practical reason: it explains why, as described in Section 6 below, a large fraction of the dashboard's 42 routes today are leftover ViralTrack-era pages (genomics, biosecurity, variants, environment, academy, phylogeny, emergency, lab-network, structural-biology) that have nothing to do with the Vivli submission's actual data. They were never deleted — per the project's own stated rule ("nothing gets silently removed") — but they had to be individually confronted, audited, and in six cases explicitly labeled as legacy/demo content so a judge browsing the site wouldn't mistake `Math.random()` output for real surveillance data. That labeling effort is documented in Section 6.5.

3. Development timeline: 46 commits over 16 days



Reading the categories left to right: the project opens with **foundation and pipeline-build** work (Jul 6–9), has its **merge/dashboard-wiring day** on Jul 10, a second and even larger **dashboard-wiring + integrity + security** push on Jul 14, more **dashboard feature work** on Jul 16, then a run of **pure correctness-fix days** (Jul 17, 18, and a big one on Jul 20) before finishing with four **submission-prep** commits on Jul 21. The category assignments above are this document's own reading of each commit's message and diff — not a field stored anywhere in git — but every commit message quoted in this report is copied verbatim from `git log`.

Two things stand out in that shape. First, correctness fixes are not front-loaded — they arrive in three separate waves (cb4c1e2 on the 14th, 38d7c7d / 434c723 / 9b049d2 on the 17th–18th, and 06dda2a / 577afed / 6f24f41 on the 20th), which means the team kept re-auditing its own output rather than treating "the pipeline runs" as "the pipeline is right." Second, the last commit before the final republish (b0ef342, "Stop tracking internal Vivli submission working files on GitHub") is itself a piece of intellectual honesty: it removed internal planning documents from public view without touching any of the published data or code, keeping the repo's public surface limited to what a judge actually needs.

4. The 30-stage analysis pipeline

4.1 What goes in

Everything starts from data obtained under **Vivli AMR Register Data Request ID 00013346**: three SOAR bacterial surveillance cohorts (SOAR 201818, 201910, 207965) and the Vivli/SENTRY antifungal surveillance cohort (2010–2024). After harmonization the isolate registry holds **34,787 records** (34,758 successfully classified — 7,836 bacterial, 26,922 fungal — plus 29 SOAR 207965 rows that don't resolve at the organism crosswalk step and are carried as unclassified rather than dropped). Every isolate can carry results for more than one drug, so the real working table is the **isolate–drug master table: 343,236 rows** spanning **59 countries, 61 organisms, and 71 drug codes** (`data/published/dataset_manifest_v1.csv`).

That surveillance data alone can't answer "why" questions, so it's joined — by ISO3 country code and year — against five external sources: WHO/World Bank life expectancy (the outcome variable), ECDC ESAC-Net antimicrobial consumption (Europe only), WHO/UNICEF Hib3 and PCV vaccination coverage, World Bank health expenditure and hospital-bed density, and IHME's Global Burden of Disease 2023 Socio-demographic Index (SDI). A sixth join, against the Global AMR R&D Hub's funding-by-country/modality export, powers the funding-vs-burden analysis in Section 5.3.

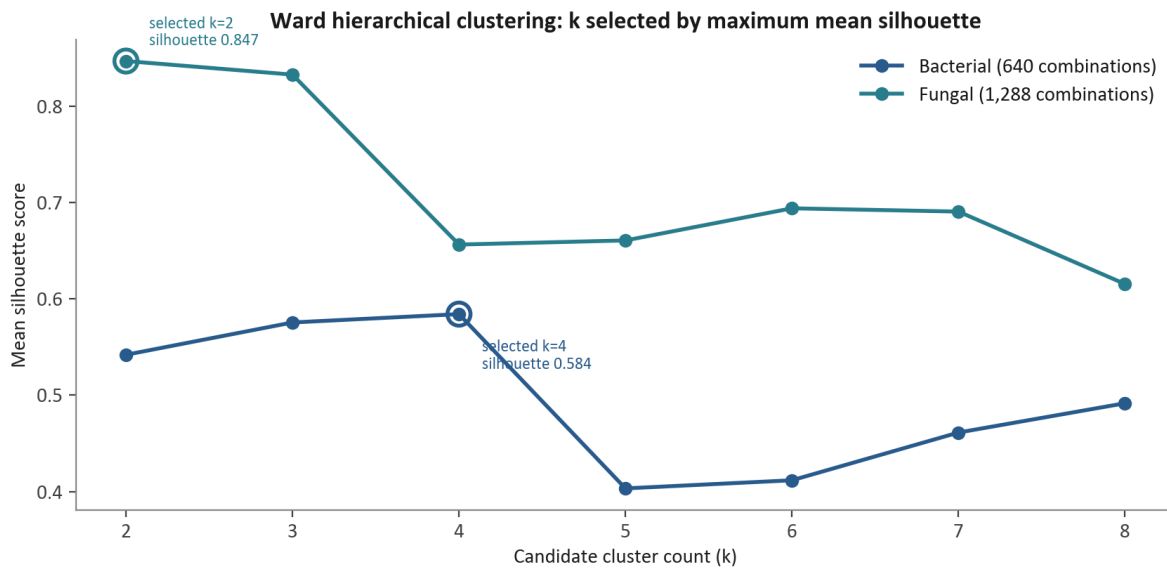
4.2 How it runs

A single orchestrator script, `analysis/run_all.py`, drives all 30 stages in order, with the stage list itself defined declaratively in `analysis/src/pipeline_registry.py`'s `FULL_PIPELINE_STAGES`. The stages fall into five phases:

1. **Preprocessing** — parse raw MIC (minimum inhibitory concentration) values, apply EUCAST breakpoints (bacterial) and CLSI/ECV three-tier classification (fungal) to turn a raw lab measurement into a susceptible/intermediate/resistant call, and build the country/organism/drug crosswalks that everything downstream depends on.
2. **Integrity** — build the identifiability ledger (which rows can and can't be classified, and why), compute Manski partial-identification bounds for anything that can't get a point estimate, and run ATLAS/PLEA calibration checks.
3. **Analytics** — descriptive profiling, MIC-trajectory / evolutionary-fitness-slope estimation, Ward hierarchical clustering into a resistance typology, the external-data join, the life-expectancy association regression, Hub funding-vs-burden alignment, and intervention-scenario scoring.
4. **Gated deliverables** — apply the `quality_gate` rule to every output row and produce the CSVs that actually get published.
5. **Verification** — re-run the evidence-gate checks (EG-01 through EG-07) and a second independent re-validation pass (J-01 through J-07) against the final deliverables, and fail loudly if anything doesn't reconcile.

4.3 Why cluster count k was chosen the way it was

The typology stage groups organism–drug–country combinations into clusters (e.g., "high burden, high trajectory") using Ward hierarchical clustering. The number of clusters, *k*, isn't picked by eye — it's chosen by scanning candidate *k* values from 2 to 8 and picking whichever maximizes the mean silhouette score (a standard measure of how well-separated the resulting clusters are). This is recorded, for every candidate *k*, in `analysis/bounds/cluster_diagnostics_v1.csv`:



Bacterial data (640 organism–drug–country combinations) peaks at **k=4** with a silhouette of **0.584**. Fungal data (1,288 combinations) peaks at **k=2** with a silhouette of **0.847** — a much cleaner split, consistent with fungal AMR typology being a coarser, less differentiated problem than bacterial typology in this dataset. Both selections are the genuine maximum across the full 2–8 sweep, not a value chosen after the fact to match a narrative.

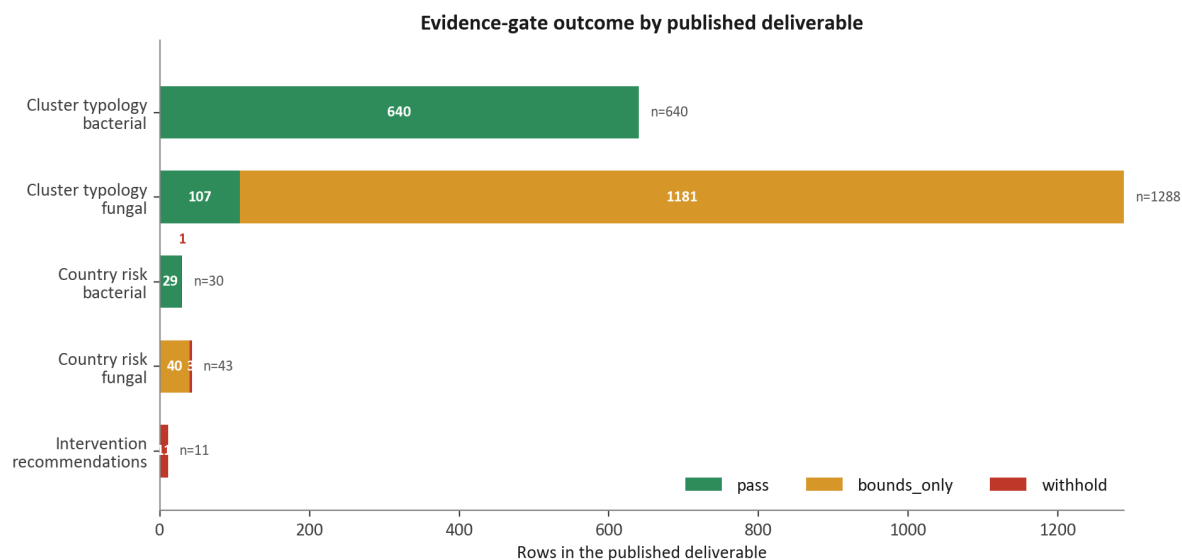
5. The evidence-gate integrity system

5.1 The problem it exists to solve

Surveillance data is incomplete in structural, not accidental, ways: a breakpoint standard might not cover a given organism–drug pair, a country might report too few isolate-years to support a stable estimate, or a genotype might only be detectable, not quantifiable. The naive approach — compute whatever the data allows and publish it — silently converts "we don't actually know" into "here's a number," and a reader has no way to tell the difference. This project's answer is a `quality_gate` field, defined as a three-value type in `analysis/src/evidence_gate_core/gate_rules.py` (`Literal["pass", "bounds_only", "withhold"]`), attached to every row of every deliverable:

- **pass** — enough independent evidence exists to publish a point estimate.
- **bounds_only** — a point estimate isn't defensible, but a worst-case range (a Manski (1989) partial-identification bound) is. The published number is a range, not a point.
- **withhold** — not even a defensible range exists yet. Nothing gets published for that row.

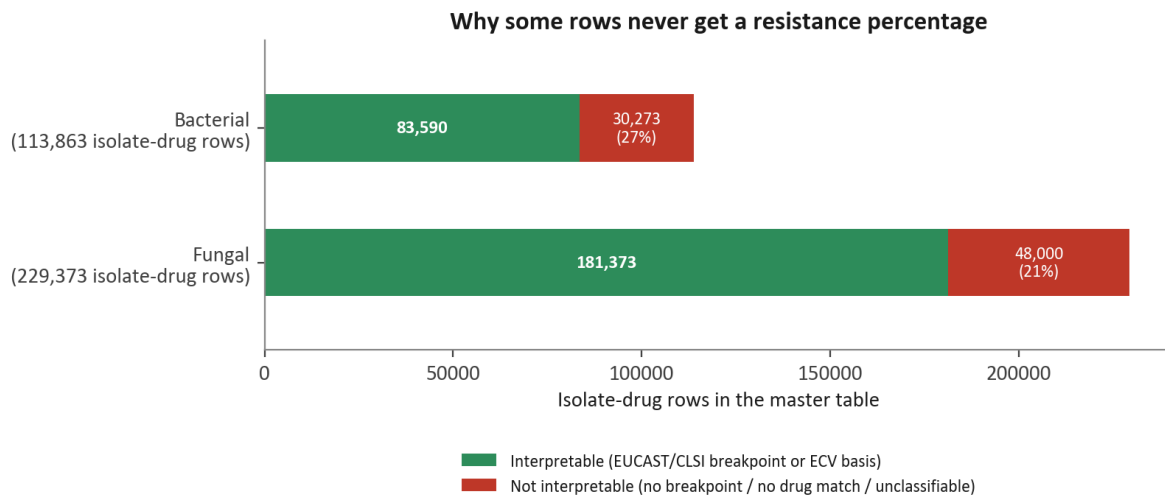
5.2 Where this shows up in the actual published data



Reading this deliverable by deliverable (`data/published/gating_comparison_v1.csv` , read directly): the **bacterial cluster typology** clears the gate completely — all 640 organism–drug–country combinations pass and are published as point estimates. The **fungal cluster typology** is the opposite story: only 107 of 1,288 combinations (8.3%) pass; the other 1,181 are published as bounds-only ranges rather than being dropped or faked. **Bacterial country risk ranking** is 29 of 30 countries passing (Ghana withheld for sparse year coverage). **Fungal country risk ranking** never reaches "pass" for any of its 43 countries — 40 are bounds-only, 3 are withheld outright — which the project treats as a finding in itself: fungal surveillance simply isn't dense enough yet to support country-level ranking claims. Most strikingly, **all 11 candidate intervention-impact rows are withheld**. The submission report is explicit about why this is a feature, not a gap: *"the dashboard therefore presents an empty ranked list with explicit reasons... so users see where evidence must improve before policy ranking."*

5.3 Why some rows never even reach a classification

Before the quality gate even runs, a row has to be *classifiable* at all — matched to a breakpoint standard and a drug code. `data/published/identifiability_ledger_v1.csv` records exactly why each of the 343,236 isolate–drug rows could or couldn't be classified:



Source: data/published/identifiability_ledger_v1.csv (classification_basis breakdown)

For bacterial rows, 73% are classifiable against an EUCAST breakpoint (either version v8.1 or the newer v10.0); the remaining 27% split between no published breakpoint existing for that organism–drug pair, no drug-code match at all, or a censored/indeterminate MIC reading. For fungal rows, 79% classify against CLSI clinical breakpoints or CLSI epidemiological cutoff values (ECVs); 21% are marked unclassifiable — not dropped, kept in the ledger with a labeled reason, which is what lets the identifiability ledger function as an audit trail rather than a silent filter.

5.4 Five bugs found and fixed in this system

The integrity layer isn't something the team built once and trusted — it was actively audited against its own live output multiple times, and real bugs were found and fixed. In order:

Bug 1 — the composite score gate bypass (fixed in 06dda2a, 2026-07-20). The gating functions in `step18b_gated_deliverables.py` correctly nulled out the `rank` field (`typology_rank`, `risk_rank`) for any row that wasn't `pass`, but never touched the underlying raw score columns (`composite_priority_score`, `composite_risk_score`, `composite_risk_score_core`) that the rank had been computed from. Those raw scores rode through to the published CSV unchanged. The dashboard's data-mapping layer then forwarded that score unconditionally. Net effect: a withheld or bounds-only country's suppressed risk rank was trivially reconstructable by anyone who sorted the dashboard's own score column — the gate was cosmetic, not structural, for that one field. The fix nulls the raw score columns alongside the rank for any non-pass row, and extends the export validator to flag any future regression of the same shape automatically.

Bug 2 — an incomplete country crosswalk silently dropping Sub-Saharan African data (fixed in 06dda2a). The ATLAS carbapenemase bound calculation matched country names against a 59-row crosswalk that had only ever been built for the SOAR/SENTRY cohorts. Six real Sub-Saharan African countries in the ATLAS data (Cameroon, Ivory Coast, Uganda, Malawi, Namibia, Mauritius) didn't match anything in that crosswalk and were silently coded `UNMAPPED` and dropped — 513 isolates removed from the SSA stratum with no warning anywhere in the output. Fixing the crosswalk (reusing the alias-and-lowercase-normalization pattern already proven correct for the GBD join) moved the affected bound's `tier1_lower` value from 15.12% to 13.16% — roughly a 13% relative shift in a number that had been silently wrong.

Bug 3 — understated uncertainty in the fungal life-expectancy model (fixed in 06dda2a). This is the most statistically substantive fix and is detailed on its own in Section 5.5 below.

Bug 4 — a fail-open default in the export validator, plus stale EUCAST parsing edge cases (fixed in 38d7c7d , 2026-07-17, and 434c723 , 2026-07-18). Two related correctness bugs: a validator function that defaulted to treating an unrecognized state as valid rather than invalid (fail-open instead of fail-closed — the wrong default for a system whose entire purpose is catching problems), and a SOAR_Hin cohort shortfall against one of the seven evidence-gate checks (EG-07) that required a clean re-run of the full pipeline to resolve.

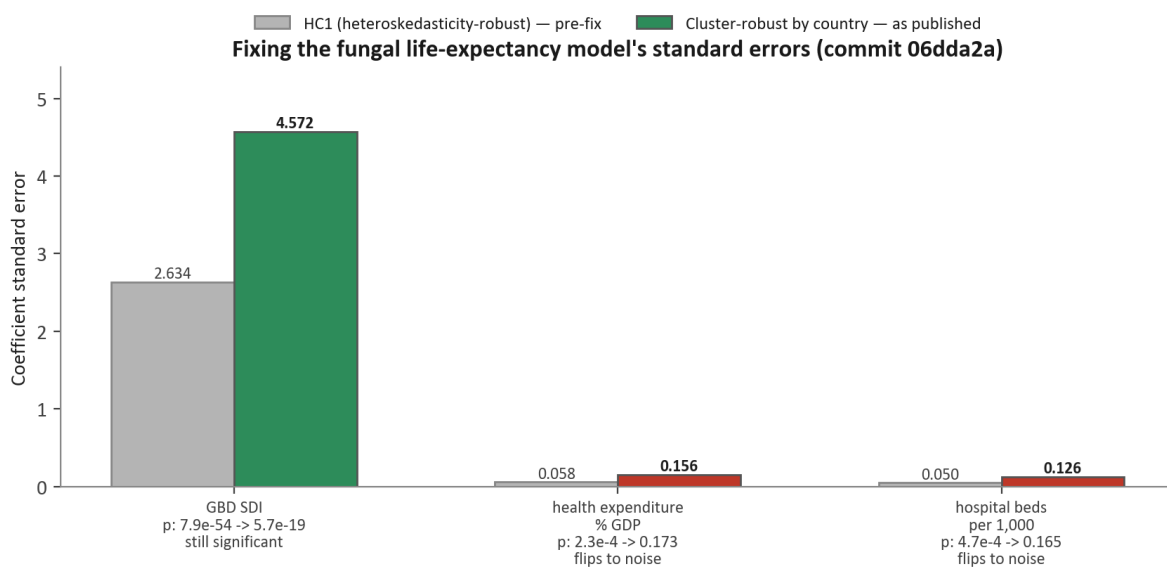
Bug 5 — a burden-pooling clip bias and an amphotericin B case-mismatch bug (fixed in 9b049d2 , 2026-07-18). A `clip(lower=1)` operation used during burden pooling was introducing a systematic bias at the boundary, and a case-sensitivity mismatch was causing amphotericin B — an important antifungal — to be excluded from a gate check it should have been included in, because the code compared a title-cased drug name against a lowercased reference list.

Two same-day, same-commit dashboard-facing bugs (577afed and 6f24f41, both 2026-07-20) are covered in Section 6.4, since they're dashboard-side manifestations of the same "does the honesty claim actually hold end-to-end" audit.

5.5 The standard-error fix, in detail

The fungal life-expectancy association model regresses life expectancy on resistance burden, trajectory, and a set of health-system covariates, using 269 country-year observations across 36 countries — meaning each country contributes roughly 7–8 repeated yearly observations. That's a panel structure, and panel data violates the independence assumption behind a standard heteroskedasticity-robust standard error (the "HC1" estimator the pipeline was originally using): repeated observations from the same country are correlated with each other, and HC1 doesn't know that. The correct approach is a **cluster-robust standard error**, clustering by country, which the pipeline now uses (`sm.OLS(...).fit(cov_type="cluster", cov_kwds={"groups": model_df["iso3_country"]})`).

The consequence of the original mistake wasn't uniform across the model's coefficients:



The GBD Socio-demographic Index coefficient's standard error widens by 74% under the correct method (2.634 → 4.572), but the coefficient (40.7) remains so large relative to even the wider error that its significance survives — p goes from 7.9×10^{-54} to 5.7×10^{-19} , both astronomically significant. **This is the one relationship in the fungal model that's genuinely robust.** Health expenditure (% of GDP) and hospital-bed density tell the opposite story: under the original HC1 method both looked like solid, sub-0.001- p confounding controls; under the correct cluster-robust method, both become statistically indistinguishable from zero ($p = 0.173$ and $p = 0.165$). Those two variables are exactly the kind of thing that would have been written into the submission as "adjusted for health-system capacity" — and that claim would have been false. Fixing the standard-error method didn't just make an existing conclusion more or less certain; it removed two conclusions that shouldn't have existed at all.

6. The dashboard: every route, what it actually does today

6.1 Stack and architecture

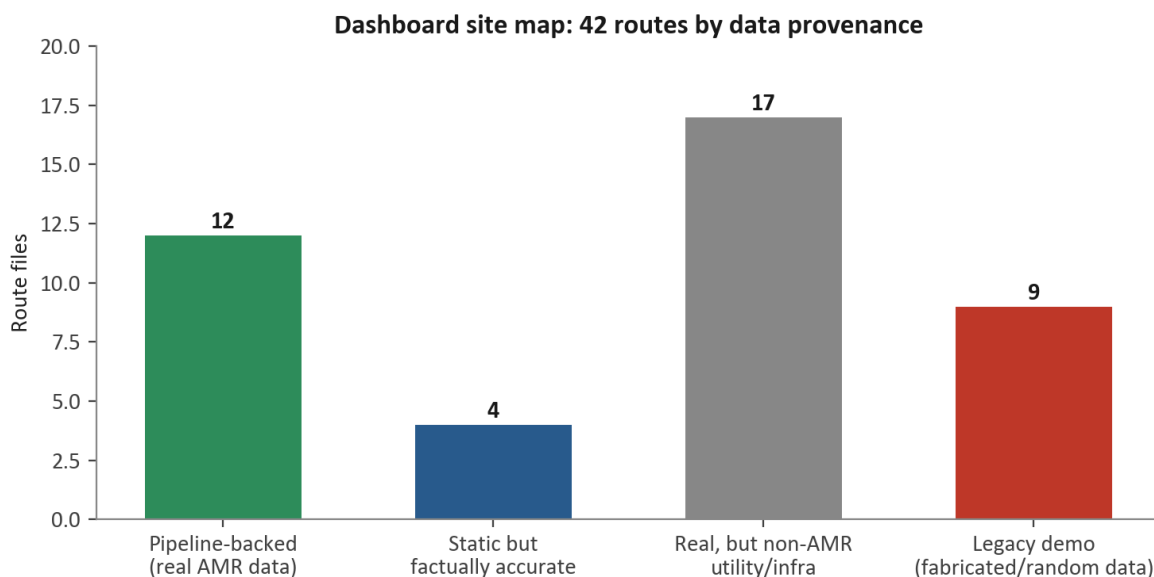
The dashboard is built on React 19 with TanStack Router/Start, TanStack Query for data fetching, Tailwind CSS 4, shadcn/Radix UI components, Recharts for charts, and Leaflet for the world map, targeting a Cloudflare deployment via `@cloudflare/vite-plugin`. Supabase is present in the stack, but only for authentication, admin tooling, and a handful of legacy features — the actual AMR data path does not touch a database at request time.

The architectural decision that matters most for judging this submission is the **static-bundle pattern**: at pipeline-publish time, the gated CSVs in `data/published/` are compiled into a single JSON file, `dashboard_bundle_v1.json`, which is copied into `dashboard/public/data/published/` and fetched directly by the client. Every AMR data page reads from that one file client-side; there is no live database query anywhere in the AMR data path. This is a deliberate simplicity choice — it means the dashboard can never show a number the pipeline didn't already gate-check, because there's no code path that queries anything else.

6.2 Navigation structure — what a visitor can actually find

The site's 42 route files are not equally discoverable. `TabNav.tsx` — the persistent primary navigation — surfaces 11 tabs: Home, Alerts, Countries, Pathogens, Lineages, Epidemiology, Marketplace, Forecasting, Policy, Methodology, and (per the header) Assistant/API Docs. `CommandPalette.tsx` (the  quick-open menu) lists 14 items, a slightly different set that adds a few utility routes. Every other route — most of the legacy ViralTrack pages, the entire `/admin/*` tree, `/auth`, `/pending`, `/uploads`, `/ingest`, `/notifications`, `/search`, and several more — exists and functions, but is reachable only by typing the URL directly or by an internal link from another page, not from primary navigation. That's consistent with the project's own README, which lists only 10 routes in its public route table and separately notes that unrelated legacy routes exist but aren't part of the submission's advertised surface.

6.3 Route inventory by what's actually behind each page



Source: direct read of every file in `dashboard/src/routes/`, this report's Section 6

Of the 42 route files that exist under `dashboard/src/routes/`, 12 are genuinely backed by the gated AMR pipeline bundle, 4 are static pages whose content was rewritten to be factually accurate (not pipeline-driven, but not fabricated either), 17 are real functioning features that simply aren't about AMR (authentication, admin tooling, generic file upload, a chat assistant, search), and 9 are leftover ViralTrack-era pages whose on-screen numbers come from `Math.random()` or hardcoded arrays rather than any real data source.

Pipeline-backed pages (real AMR data, read from `dashboard_bundle_v1.json`):

Route	What it actually shows today
<code>/</code> (index)	Homepage KPI tiles (total isolates, distinct countries, high-risk count, average predicted intervention gain) and a summary of the gated headline findings. The "predicted gain" tile was a fabrication bug until <code>577afed</code> — see 6.4.
<code>/alerts</code>	Rule-based alerts derived from the gated country-risk and typology data (e.g. <code>high_burden_high_trajectory</code> combinations), not a live feed.
<code>/countries</code>	Per-country resistance summary table with the same gate-aware "Avg gain" and country-count logic fixed in <code>577afed</code> / <code>6f24f41</code> .
<code>/pathogens</code>	Organism-level breakdown; this is the page where a fungal gate-bypass bug (unsorted slice showing bounds-only rows as if they were precise percentages) was fixed in <code>6f24f41</code> .
<code>/lineages</code>	Organism–drug typology browser (the cluster output from Section 4.3), gate-aware.
<code>/epidemiology</code>	Trajectory and burden views over the same gated typology data.
<code>/policy</code>	Intervention-scenario view — reachable only via the homepage CTA or <code>⌘K</code> , not primary TabNav, and correctly shows the empty/gated intervention list described in 5.2.
<code>/marketplace</code>	The Hub funding-vs-burden explorer — Q3 of the submission's guiding questions, live-wired to <code>hub_funding_composition_summary_v1.csv</code> and <code>funding_gap_summary_v1.csv</code> as of <code>f1d9bfc</code> / <code>0fcd941</code> (2026-07-14/16).
<code>/forecasting</code>	A trajectory/forecast view over real gated data; the README marks the concept as legacy-named but the data underneath is real.

Route	What it actually shows today
<code>/methodology</code>	The evidence-gate explainer page — describes the pass/bounds_only/withhold system in Section 5 to a lay visitor.
<code>/news</code>	Rewired (per this session's plan) to show real pipeline-derived system-activity events — gate-status changes, republish timestamps — rather than the original fabricated wire-service headlines.
<code>/reports</code>	Export/report-generation view over the gated bundle.

Static-but-honest pages (rewritten off fabricated ViralTrack content, per Plan Phase B):

Route	Status
<code>/about</code>	Rewritten to describe the actual Manski-bounds + evidence-gated pipeline, EUCAST/CLSI classification, and GBD/SDI joins, replacing the original ViralTrack stat tiles ("47 countries / 72 sentinel sites / 8.6M people surveilled") and a fabricated partnerships claim (Africa CDC, NICD, etc., with no supporting basis anywhere in the repo) that was approved for removal and replaced with an honest data-sources/acknowledgments list.
<code>/api-docs</code>	Rewritten to document the real published bundle schema and how to read the underlying CSVs directly, replacing six fabricated REST endpoints against a fictitious <code>api.amr-lifeintel.org</code> .
<code>/team</code>	Now states the actual submission team.
<code>/partnerships</code>	Honest data-sources listing.

Real functionality, unrelated to AMR (utility/infrastructure/legacy-but-truthful):

`/assistant` (a working chat assistant — see 6.6), `/search`, `/uploads`, `/login`, `/auth`, `/pending`, `/notifications`, `/ingest`, `/lab-network`, `/structural-biology`, and the seven `/admin/*` routes (`admin`, `admin.index`, `admin.dashboard`, `admin.audit`, `admin.roles`, `admin.security`, `admin.users`). These pages do real work — authentication gating, generic file storage, admin user/role management — but none of it is AMR-specific; they're infrastructure the ViralTrack template came with that the team kept because it functions correctly, just isn't part of the submission's scientific content.

6.4 Two dashboard-side honesty bugs, fixed the same day as the pipeline fixes (577afed , 6f24f41 , both 2026-07-20)

These matter because they show the "no fabricated numbers" discipline extended past the pipeline and into the presentation layer, where a bug can undo an upstream fix.

The homepage "Predicted gain" fabrication. The homepage KPI computation filtered the 11 intervention rows to only the positive-valued ones and averaged them — with no `quality_gate` check and no minimum-sample-size floor. Given that all 11 intervention rows are withheld (Section 5.2), the homepage was still showing "+0.22y" — a number averaged from a single withheld data point, formatted with a hardcoded "+" sign as if it were a confident finding. The fix reused an existing, already-correct gating pattern from the country-map code (`MIN_INTERVENTION_SAMPLES = 3`, `quality_gate === "pass"` check) and made the value render as "—" with an explicit "not enough measured interventions yet" caption when the gate isn't cleared, instead of silently returning `0` or a stale positive number.

The "Countries" KPI overcount. Two places (`LiveAMRWorldMap.tsx` and `countries.tsx`) computed a "Countries" statistic as a raw row count — but the underlying data has one row per (country, pathogen-type) pair, not one row per country. With the pathogen filter at its default "all" setting, that summed to 30 bacterial + 43 fungal = 73 "countries," a 28% overcount against the true 57 distinct countries. The fix applies `new Set(rows.map(r => r.iso3)).size` — the same pattern the homepage KPI function already used correctly — at both sites.

The `/pathogens` gate bypass. A slice of fungal rows used to populate the pathogens page wasn't sorted before being sliced, which meant bounds-only rows could end up displayed as if they were precise pass-gated percentages. Fixed alongside the two bugs above in `6f24f41`.

6.5 Legacy-demo labeling: `/genomics`, `/biosecurity`, `/variants`, `/environment`, `/academy`, `/phylogeny`

These six routes are the deepest ViralTrack-era holdovers — pages about genomic sequencing surveillance, biosecurity threat monitoring, viral variant tracking, environmental (wastewater) monitoring, and a training academy, none of which this AMR submission has any underlying capability to back with real data. Rather than delete them (against the project's standing "nothing removed without approval" rule) or leave them silently displaying `Math.random()`-generated numbers as if real, each was given a shared, reusable `LegacyDemoBanner` component that visibly states the page is legacy/demo content from a prior product direction and is not part of the Vivli 2026 AMR submission. The routes stay reachable — including the `/search` → `/genomics` cross-link — but a visitor can no longer mistake their content for submission data. As of the most recent audit, four further routes with similar fabricated-content characteristics (`/forecast-compare`, `/emergency`, `/lab-network`, `/intel/$signal`) had not yet received the same banner treatment — they remain functionally real-but-unrelated or fabricated-but-unlabeled, a known gap rather than an oversight, tracked in the standing project plan.

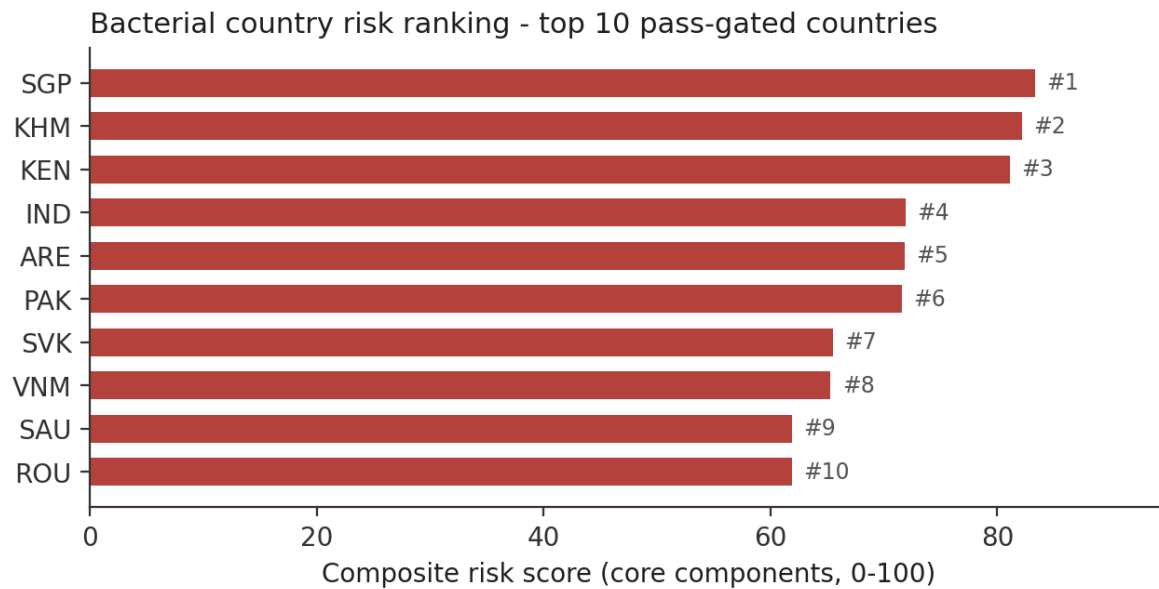
6.6 The one server-side feature: `/api/chat`

`dashboard/src/routes/api/chat.ts` implements a real streaming chat endpoint (Vercel AI SDK's `streamText`, tool-calling enabled) proxied through the Lovable AI Gateway to a Gemini model. It exposes two callable tools that query the real published bundle — meaning the assistant can answer a question like "which countries pass the risk-ranking gate" by actually reading `dashboard_bundle_v1.json`, not by hallucinating from training data. This is the one place in the dashboard where a language model sits directly in the request path, and it's scoped narrowly to the same gated data everything else uses.

7. What the pipeline actually found (Results, Q1–Q5)

The Vivli submission report (`docs/VIVLI_SUBMISSION_REPORT_v1.md`, read directly, verified against pipeline run `20260720T144743`) organizes results around five guiding questions:

Q1 — Which resistance profiles co-occur with the lowest life expectancy? Full bacterial typology releases 640/640 combinations (moderate: 387, high_trajectory: 92, high_burden: 92, high_burden_high_trajectory: 69). Top dual-high examples include *S. pneumoniae*/cefaclor in Slovakia and Cambodia, and *H. influenzae*/trimethoprim-sulfamethoxazole in Morocco. Bacterial country risk ranking clears 29/30 countries, led by Singapore, Cambodia, Kenya, India, and UAE. Fungal country risk never reaches pass status for any of its 43 countries.

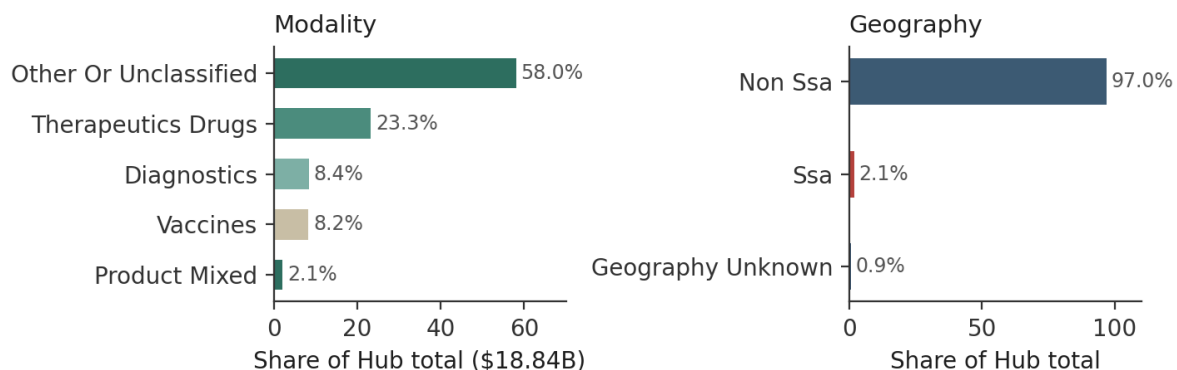


Source: country_risk_ranking_bacterial_gated_v1.csv. Ghana withheld (missing_country_gate); not shown.

Q2 — What drives the relationship (consumption, health-system capacity, vaccination, hospital exposure)? The fungal model (n=269, 36 countries, $R^2=0.638$) is dominated by GBD SDI (coefficient 40.7, cluster-robust SE 4.57, $p=5.7 \times 10^{-19}$) — dropping SDI collapses R^2 to 0.175. As established in Section 5.5, health expenditure and hospital-bed density are not distinguishable from zero once standard errors are computed correctly. The bacterial primary model (n=16, 5 countries) is explicitly flagged small-sample and excluded from coefficient claims. Antimicrobial consumption data covers only 21 bacterial Europe-only rows; no antifungal consumption series or hospital-acquired-exposure series exists in the joined data at all.

Q3 — Where does R&D investment concentrate relative to surveillance burden? All four bacterial organisms studied show burden share far exceeding matched Hub funding share — *K. pneumoniae* at 32.3% of burden vs. 1.7% of funding (a 30.6 percentage-point gap), *E. coli* at 28.4% vs. 3.2%. Of the Global AMR R&D Hub's tracked \$18.84B portfolio, only 2.1% is geographically matched to Sub-Saharan Africa versus 97.0% elsewhere, and by modality: 23.3% therapeutics, 8.4% diagnostics, 8.2% vaccines, 58.0% other/unclassified.

Global AMR R&D Hub funding composition



Source: hub_funding_composition_summary_v1.csv. Hub excludes private/VC funding.

Q4 — Which combinations are early-warning candidates (not yet high-burden but rising fast)? 92 of 640 bacterial combinations are `high_trajectory`; 30 of the 107 pass-gated fungal combinations are as well — the pipeline's proposed early-intervention target list, distinct from the already-severe dual-high group.

Q5 — Which intervention category would yield the largest life-expectancy gain? None of it. Of 11 candidate intervention rows, zero currently clear the gate for a public ranking (Section 5.2), each carrying an explicit reason (small sample, data gap, funding-only proxy without a measured life-expectancy elasticity, or fungal vaccination excluded from scope by design).

8. The submission itself

`docs/VIVLI_SUBMISSION_REPORT_v1.md` is a five-page report (references excluded from the page count, per Vivli's rules) targeting Data Request ID 00013346, built and polished across three commits on 2026-07-21 (`9482321` republishing pipeline outputs and syncing the README/team page, `5827dd2` adding the report and its two figures, `69e2329` a final polish pass). The report's two embedded figures — the bacterial country-risk ranking and the Hub funding-composition breakdown — are generated by `docs/figures/generate_report_figures.py` directly from the same published CSVs the dashboard reads, so the report and the live dashboard cannot drift apart from each other; they're two views of one data source. The final commit on `main`, `b0ef342`, removed internal working documents from public GitHub visibility without touching any published/submitted content.

9. What isn't finished

This project describes its own limitations rather than hiding them, and this report inherits that discipline. As of 2026-07-22: four dashboard routes with fabricated-or-unrelated content (`/forecast-compare`, `/emergency`, `/lab-network`, `/intel/$signal`) don't yet carry the legacy-demo banner that six sibling routes already have. Most organism–drug strata across the full dataset — 1,022 of 1,822, or 56.1% — are withheld outright, not because of a pipeline defect but because the underlying surveillance data genuinely doesn't support a claim yet; the identifiability ledger in Section 5.3 documents exactly why. The bacterial life-expectancy model is too small ($n=16$) to support coefficient-level claims. Antimicrobial-consumption data is Europe-only, and no hospital-acquired-exposure series exists in any joined dataset. None of these are secret — they're stated in the submission report's own Limitations section and in the identifiability ledger any visitor can open.